
LAPORAN PRAKTIKUM INTERNET OF THINGS II



Disusun Oleh:

Nama: M Riki Pratama

Kelas: Teknologi Rekayasa Internet (3C)

NPM: 25782072

Dosen:

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG 2026

Modul 4: Komunikasi Asinkron dan WebSockets

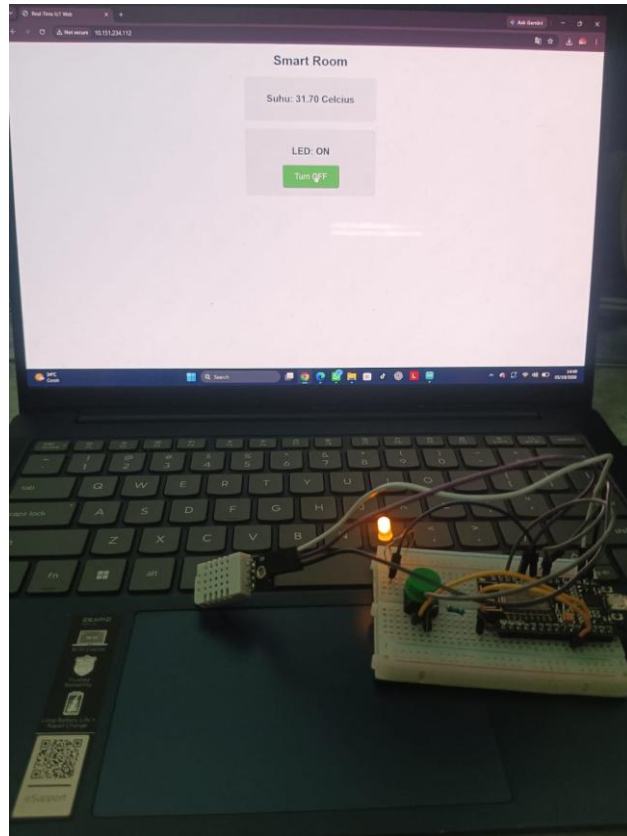
4.5 Praktikum

4.5.1 Praktikum 1: *Real-Time Dashboard & Sinkronisasi Tombol Fisik*

Skenario: Anda memiliki lampu LED (simulasi aktuator/lampu cerdas) yang bisa dihidupkan melalui sakelar fisik (*push button*) di *breadboard*, ATAU melalui tombol di antarmuka web *smartphone* Anda. Keduanya harus tersinkronisasi dua arah secara *real-time*. Jika Anda menekan tombol fisik di meja praktikum, status visual indikator di antarmuka *smartphone* harus otomatis berubah seketika, begitu pula sebaliknya jika dikontrol dari web.

Langkah Kerja Rangkaian:

1. Hubungkan kaki **Anoda** (kaki panjang) LED ke pin **D6 (GPIO 12)**. Hubungkan kaki **Katoda** (kaki pendek) LED ke salah satu kaki **Resistor 220 Ohm**, lalu hubungkan kaki resistor lainnya ke pin **GND** NodeMCU (*sama seperti rangkaian LED pada Modul 3*).
2. Hubungkan pin **DATA / OUT** DHT ke pin **D4 (GPIO 2)**. (VCC DHT ke 3V3, GND ke GND).
3. Pasang **Push Button** pada *breadboard*, melintasi parit tengah. Hubungkan salah satu kakinya ke jalur **3V3 (3.3V)**. Hubungkan kaki seberang tombol ke pin **D2 (GPIO 4)** NodeMCU. Pada titik kaki yang terhubung ke pin D2 tersebut, pasang salah satu kaki **Resistor 10k Ohm**, lalu hubungkan ujung kaki resistor lainnya ke jalur **GND** (*konfigurasi resistor pull-down eksternal seperti pada Modul 1*).



Langkah Kerja Pemrograman: Kode program ini cukup kompleks karena mengombinasikan *Front-End* (HTML, CSS, JavaScript) dengan *Back-End* (C++). Agar *real-time response* tombol tidak terganggu oleh sensor DHT yang butuh jeda baca lama, kita menerapkan teknik pemrograman *non-blocking* menggunakan fungsi `millis()`.

1. Buat berkas baru (*New Sketch*). Tuliskan kode program berikut.

```

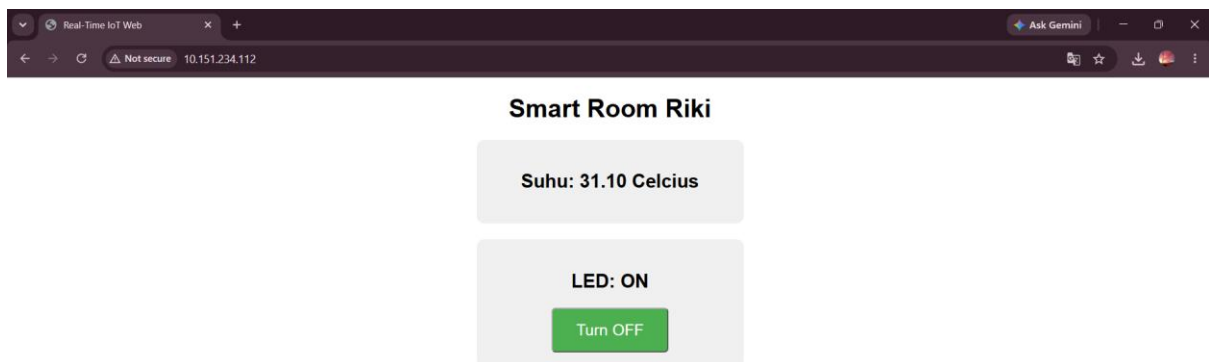
sketch_oct5a | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...)
sketch_oct5a.ino
35 <title>Real-Time IoT Web</title>
36 <style>
37   body { font-family: Arial; text-align: center; }
38   .card { background: #f0f0f0; margin: 20px auto; padding: 20px; max-width: 300px; border-radius: 10px; }
39   button { padding: 15px 30px; font-size: 20px; border-radius: 5px; cursor: pointer; color: white; }
40   .btn-on { background-color: #4CAF50; }
41   .btn-off { background-color: #f44336; }
42 </style>
43 </head>
44 <body>
45   <div>Smart Room Riki</div>
46   <div class="card">
47     <div>
48       <span id="tempValue">...</span> Celcius</div>
49     </div>
50     <div class="card">
51       <div>LED: <span id="ledStatus">OFF</span></div>
52       <button id="toggleBtn" class="btn-off" onclick="toggleLed()">Turn ON</button>
53     </div>
54   </div>
55   <script>
56     // Membuka terowongan koneksi websocket ke alamat IP ESP8266
57
Output Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7')
New Line 115200 baud
*****
IP Address: 10.151.234.112
.....
IP Address: 10.151.234.112
Client Websocket #1 terhubung
Client Websocket #1 terputus
Client Websocket #2 terhubung
.....
IP Address: 10.151.234.112
Client Websocket #1 terhubung
...
IP Address: 10.151.234.112
Client Websocket #1 terhubung
Ln 47, Col 32 NodeMCU 1.0 (ESP-12E Module) on COM7 14:59 05/10/2026

```

2. Upload program. Pastikan IP Address didapatkan dari Serial Monitor.

3. Buka IP Address tersebut di *browser* laptop Anda, dan **sekaligus** buka IP yang sama di *browser* ponsel (*smartphone*) Anda secara berdampingan.
4. **Uji 1 (Kendali Web Tersinkronisasi):** Klik tombol pada *browser* laptop Anda. Perhatikan bagaimana *browser* di ponsel Anda ikut berubah (*Turn ON*) secara presisi, dan lampu LED fisik ikut menyala!
5. **Uji 2 (Kendali Perangkat Keras):** Sentuh *push button* mekanik di meja Anda. Amati bahwa antarmuka *browser* di laptop maupun ponsel Anda secara serentak berganti status (*Turn OFF* menjadi *Turn ON*) **tanpa** terjadi proses *loading* atau *refresh* HTML halaman sama sekali! Kecepatan respon seketika ini adalah manfaat dari pemrograman *Asynchronous* dan pemisahan (*decoupling*) pembacaan memori sensor DHT.

Hasil:



4.6 Latihan

Kerjakan latihan berikut untuk menguji pemahaman Anda terhadap arsitektur web modern!

- **Latihan 1 (Teori Jaringan):** Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

Jawaban: Teknologi *WebSockets* jauh lebih hemat konsumsi data/bandwidth karena menggunakan koneksi persisten (*persistent connection*) dua arah (*full-duplex*) melalui satu kali proses *handshake* awal. Data baru dikirimkan secara *real-time* sesuai perubahan kondisi tanpa header tambahan. Sebaliknya, metode *Meta Refresh HTTP* memaksa browser untuk mengunduh ulang seluruh halaman web secara berkala, yang menyebabkan overhead lalu lintas data yang besar, pemborosan bandwidth, dan membuat layar berkedip (*flicker*) setiap kali halaman dimuat ulang.

- **Latihan 2 (Analisis Waktu/Timer):** Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan *NodeMCU*) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

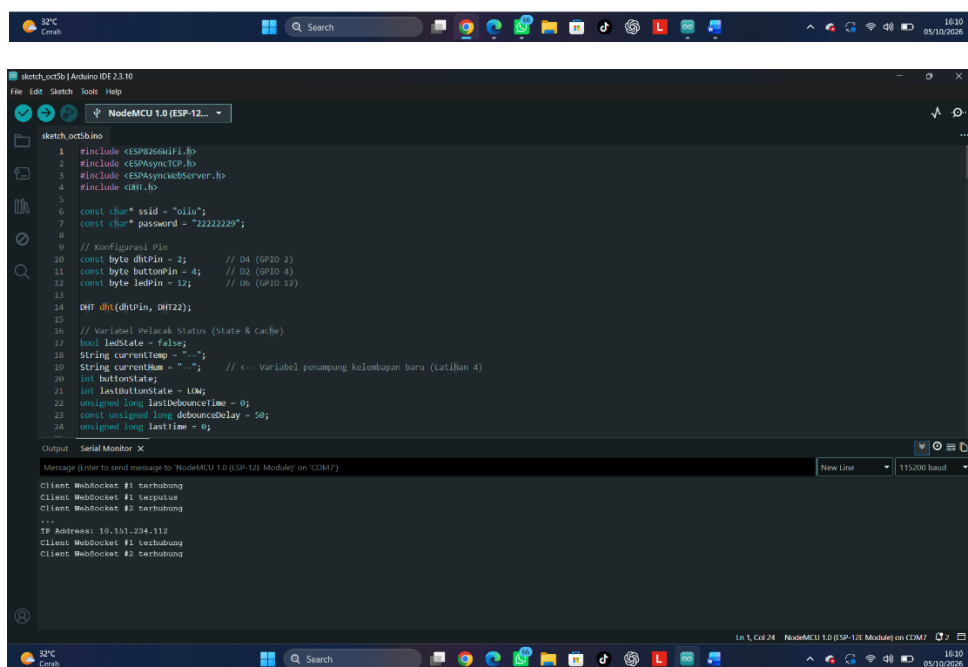
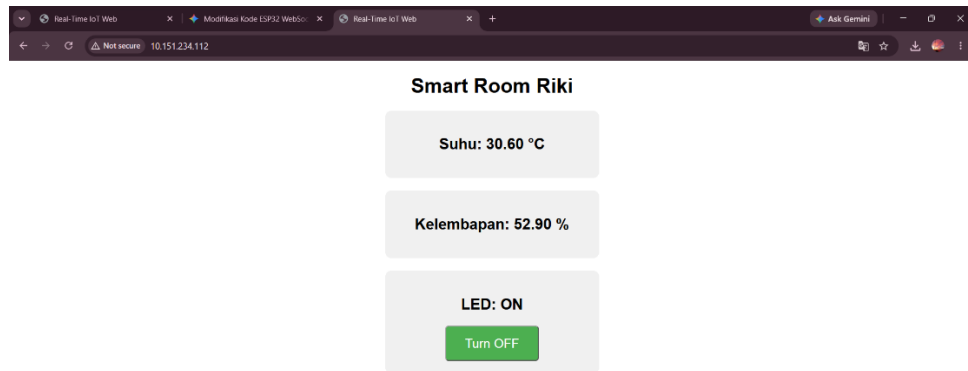
Jawaban: Fungsi `delay(3000)` bersifat *blocking* (menghentikan seluruh siklus eksekusi program mikroprosesor selama 3 detik). Jika dipasang di dalam `void loop()`, ESP8266/NodeMCU tidak akan dapat melayani lalu lintas jaringan *WebSocket*, memproses siklus TCP, atau merespons permintaan masuk dari browser selama jeda waktu tersebut, sehingga berisiko menyebabkan koneksi terputus (*timeout*) dan sistem menjadi macet atau tidak responsive.

- **Latihan 3 (Modifikasi Front-End):** Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML (`<span id="...">`) apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

Jawaban: Berdasarkan arsitektur antarmuka web *WebSocket*, elemen HTML dengan atribut `id="..."` yang menjadi target injeksi atau pembaruan teks oleh skrip JavaScript biasanya meliputi elemen penampil status perangkat dan data sensor, seperti `id="ledStatus"` untuk status lampu dan `id="tempValue"` untuk nilai suhu.

- **Latihan 4 (Pengembangan Telemetry Sensor - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via *WebSockets*:
 - i. **Sisi Back-End (C++):** Buat variabel penampung (misal: `String currentHum = "--";`). Baca nilai kelembapan menggunakan fungsi `dht.readHumidity()` di dalam siklus timer *non-blocking* `millis()`. Perbarui fungsi `notifyClients()` agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: `{"led":"...", "suhu":"...", "hum":"..."}`).

- ii. **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu (<div class="card">) baru pada kerangka HTML sebagai penampil kelembapan (misal: <h2>Kelembapan: -- %</h2>). Kemudian, modifikasi fungsi JavaScript onMessage(event) agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen humValue.
- Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

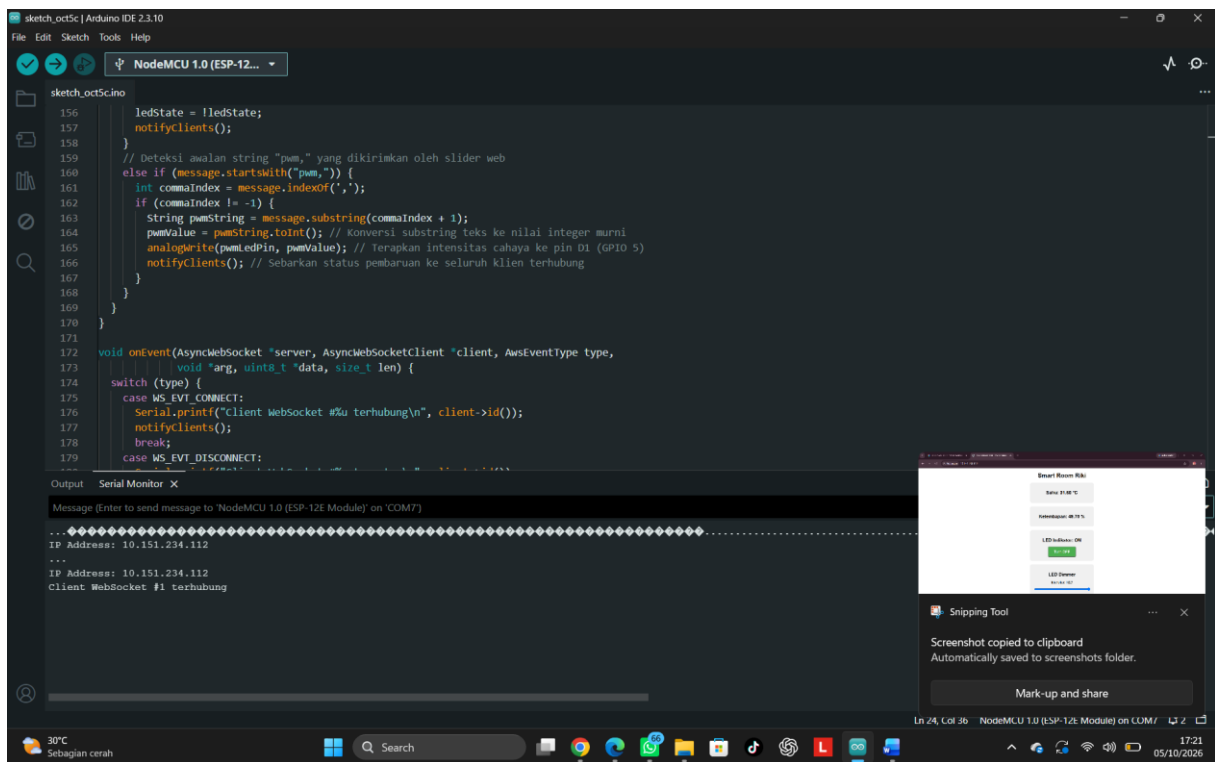
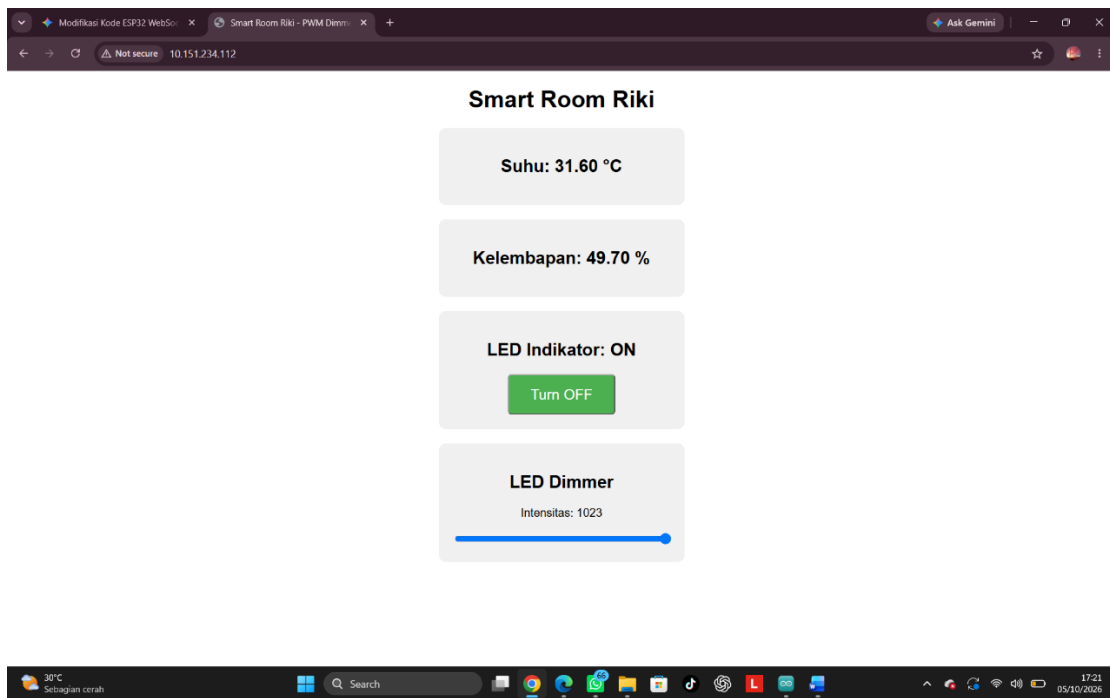


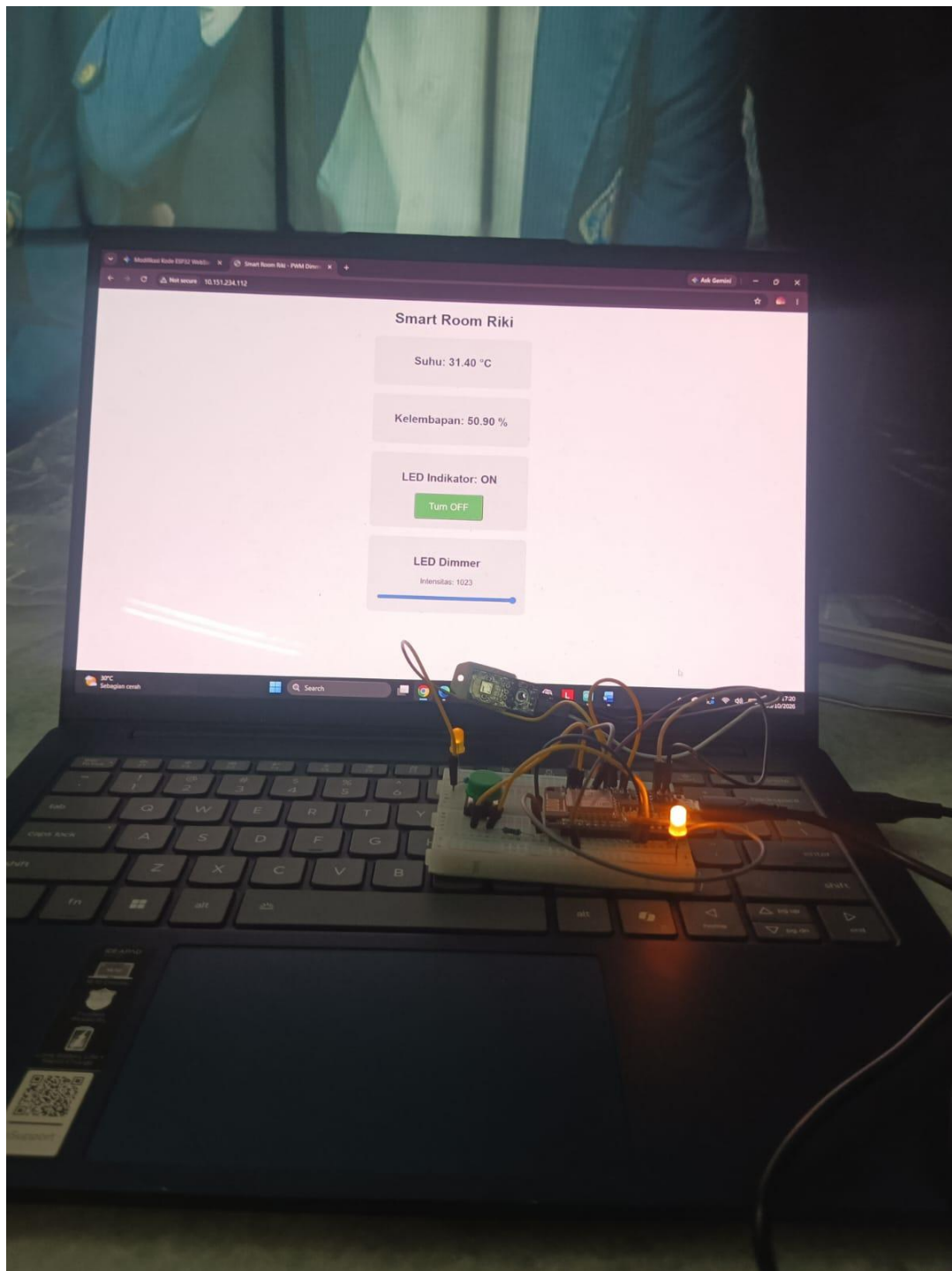
4.7 Tugas: Sistem Kendali Intensitas Cahaya (PWM Slider) via *WebSockets*

Sinyal Digital (*ON/OFF*) sangat berguna, namun bagaimana jika Anda ditugaskan merancang antarmuka penyesuaian intensitas redup-terang lampu (*Dimmer*) dari *smartphone*?

Skenario Proyek (*Smart Lighting*): Anda akan mengeksplorasi penggunaan modulasi sinyal listrik `analogWrite()` (atau dikenal dengan fitur PWM) dan memadukannya dengan teknologi masukan elemen *Slider / Range* dari HTML5.

- **Tujuan:** Menguasai transmisi paket *String* WebSocket berkelanjutan (*continuous packet*) dari manipulasi antarmuka layar *slider* untuk merombak parameter persentase daya komponen aktuator.
- **Tingkat Kesulitan:** Lanjut
- **Instruksi Perangkaian:**
 - i. Tambahkan sebuah komponen **LED** yang dilengkapi **Resistor 220 Ohm** (sebagai pengaman arus).
 - ii. Hubungkan kaki Anoda (positif) sistem LED tersebut ke pin **D1 (GPIO 5)**.
- **Instruksi Pemrograman (Front-End & Back-End):**
 - i. Modifikasi desain kerangka web HTML Anda. Sisipkan elemen input *Slider* HTML. (*Hint sintaksis: `<input type="range" min="0" max="1023" id="pwmSlider" onchange="sendPWM(this.value)">`*)
 - ii. Buat sebuah fungsi JavaScript bernama `sendPWM(value)` yang akan mengirimkan paket *string* teks berformat khusus (misalnya "pwm,512") ke NodeMCU melalui *websocket*.
 - iii. Pada ruang fungsi C++ `handleWebSocketMessage()` di sisi server (NodeMCU), pecah (*parsing*) atau periksa string tersebut menggunakan fitur deteksi *substring* atau bahasa pemrosesan C++ lainnya.
 - iv. Jika Server NodeMCU sukses mendeteksi dan mengambil angka murninya, panggil fungsi aktuator `analogWrite(pinLED, nilaiPWM)` untuk secara elegan meredupkan atau menerangkan LED LED fisik secara seketika sesuai gesekan layar Anda!
- **Kriteria Keberhasilan:**
 - Di layar gawai Anda, hadir tuas gulir horizontal (*Slider*).
 - LED di dunia nyata dapat diredupkan dan diterangkan secara bertahap saat Anda menggeser tuas *slider*, bebas dari lag (*real-time response*).
 - Kendali LED dan pembacaan Sensor *DHT* bawaan Praktikum 1 dipastikan masih bekerja berdampingan dengan sempurna.





Modul 5: Topologi Jaringan Terdesentralisasi (ESP-NOW dan Mesh)

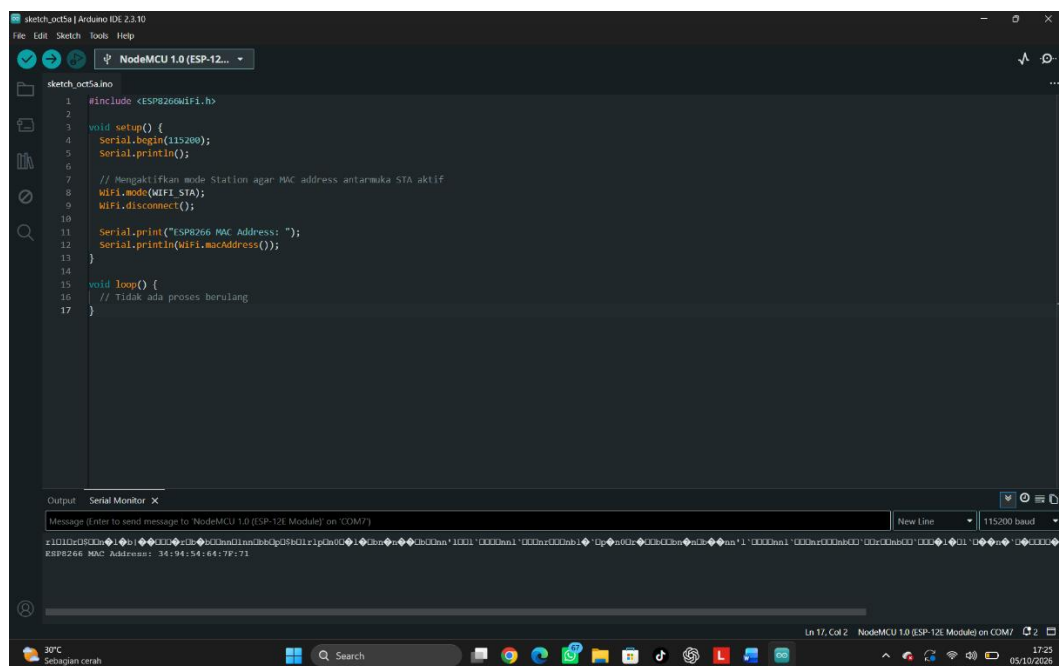
5.5 Praktikum

5.5.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU

Sebelum membangun jaringan ESP-NOW, setiap anggota kelompok wajib membaca dan mencatat alamat fisik (*MAC Address*) dari masing-masing *board* NodeMCU.

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*) pada Arduino IDE.
2. Tuliskan kode program pembaca MAC Address berikut:



```
1 #include <ESP8266WiFi.h>
2
3 void setup() {
4   Serial.begin(115200);
5   Serial.println();
6
7   // Mengaktifkan mode Station agar MAC address antarmuka STA aktif
8   WiFi.mode(WIFI_STA);
9   WiFi.disconnect();
10
11   Serial.print("ESP8266 MAC Address: ");
12   Serial.println(WiFi.macAddress());
13 }
14
15 void loop() {
16   // Tidak ada proses berulang
17 }
```

Output: Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on COM7)

ESP8266 MAC Address: 34:94:54:64:7F:71

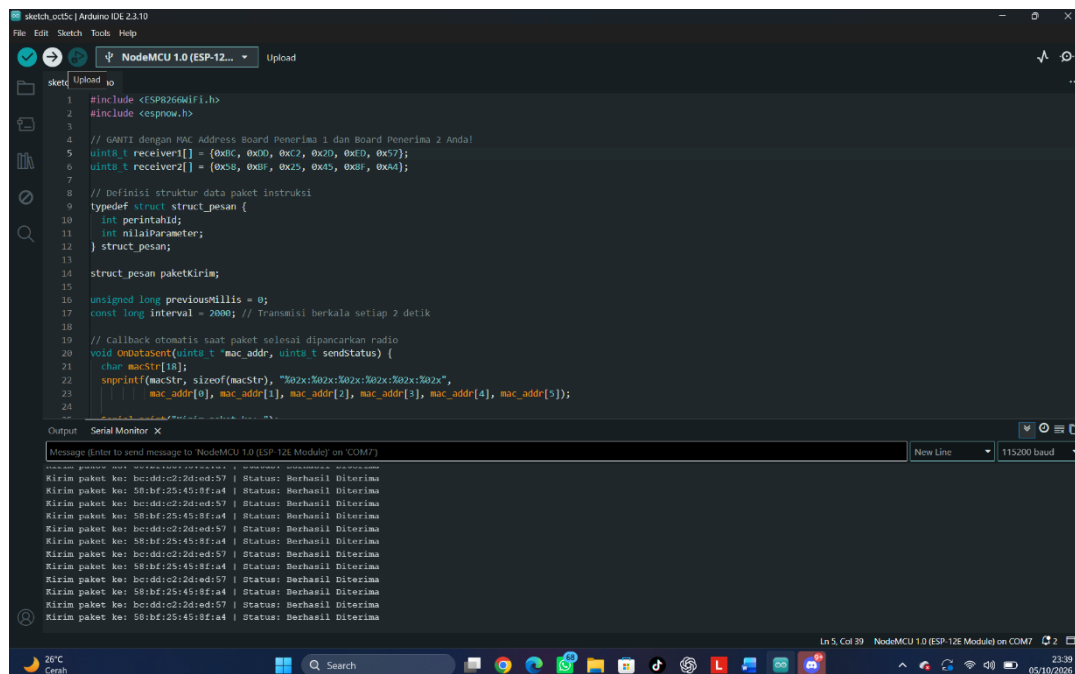
3. **Upload** program ke masing-masing *board* NodeMCU dalam kelompok Anda secara bergantian.
4. Buka **Serial Monitor** pada kecepatan **115200 baud**, lalu tekan tombol fisik **RST** pada *board*.
5. Catat 6 pasang digit heksadesimal yang muncul di layar (contoh: BC:FF:4D:23:45:90).
6. Beri label fisik pada *board* (misal: "Node 1", "Node 2", "Hub Pusat") beserta alamat MAC masing-masing agar tidak tertukar saat perakitan.

5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

Skenario: Satu unit NodeMCU bertindak sebagai Controller / Sender, menyiarkan paket data instruksi yang sama secara langsung ke dua unit NodeMCU lain yang bertindak sebagai Slave / Receiver tanpa perantara router.

Langkah Kerja Pemrograman Sisi Pengirim (Controller):

1. Masukkan alamat MAC milik Board Penerima 1 dan Board Penerima 2 ke dalam variabel receiver1 dan receiver2.
2. Format tanda titik dua (:) diubah menjadi bilangan heksadesimal berawalan 0x yang dipisahkan oleh tanda koma (contoh: BC:FF:4D:23:45:90 → {0xBC, 0xFF, 0x4D, 0x23, 0x45, 0x90}).
3. Tuliskan kode berikut dan unggah ke NodeMCU 1:



```
1 #include <ESP8266WiFi.h>
2 #include <ESPnow.h>
3
4 // GANTI dengan MAC Address Board Penerima 1 dan Board Penerima 2 Anda!
5 uint8_t receiver1[] = {0xC1, 0x00, 0x02, 0x20, 0xED, 0x57};
6 uint8_t receiver2[] = {0xC8, 0x00, 0x25, 0x45, 0x0F, 0xA4};
7
8 // Definisi struktur data paket instruksi
9 typedef struct struct_pesan {
10     int perintah;
11     int nilaiParameter;
12 } struct_pesan;
13
14 struct_pesan paketKirim;
15
16 unsigned long previousMillis = 0;
17 const long interval = 2000; // Transmisi berkala setiap 2 detik
18
19 // Callback otomatis saat paket selesai dipancarkan radio
20 void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
21     char macStr[18];
22     sprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x",
23             mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
24 }
25
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7')

Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima

Kirim paket ke: 58:bf:25:45:8f:a4 | Status: Berhasil Diterima

Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima

Kirim paket ke: 58:bf:25:45:8f:a4 | Status: Berhasil Diterima

Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima

Kirim paket ke: 58:bf:25:45:8f:a4 | Status: Berhasil Diterima

Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima

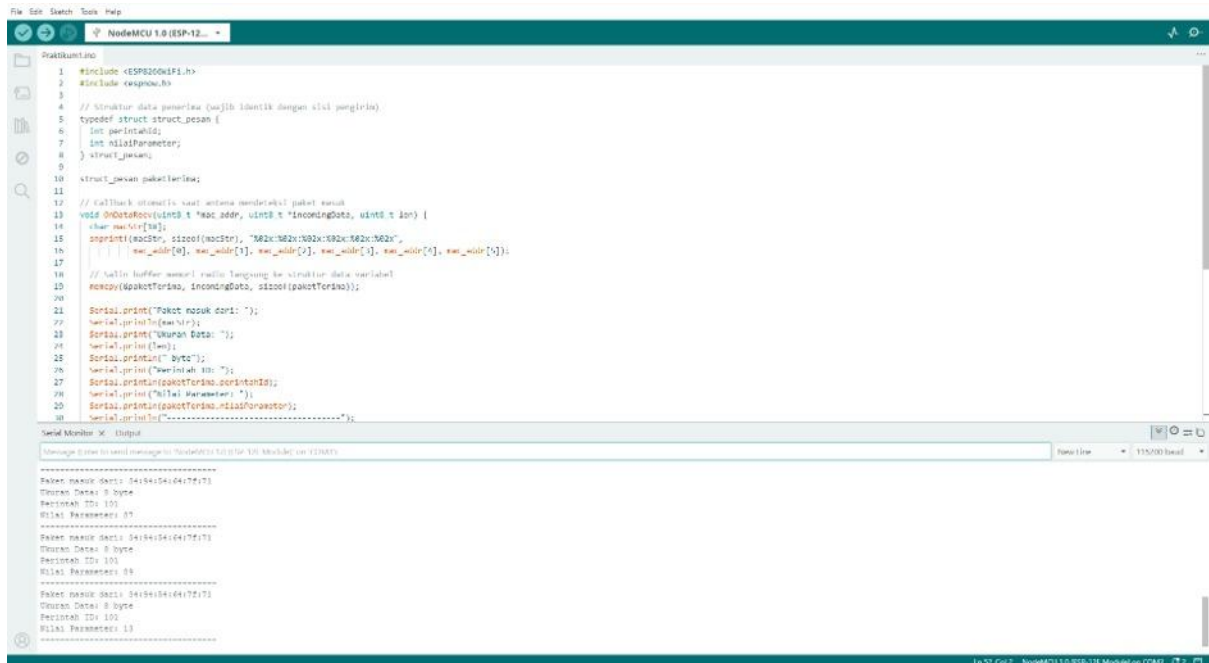
Kirim paket ke: 58:bf:25:45:8f:a4 | Status: Berhasil Diterima

Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima

Kirim paket ke: 58:bf:25:45:8f:a4 | Status: Berhasil Diterima

Langkah Kerja Pemrograman Sisi Penerima (Slave):

1. Unggah kode sketsa berikut ke **NodeMCU 2** dan **NodeMCU 3**.
2. Board penerima tidak perlu mengetahui alamat MAC milik Controller.



```
1 #include <ESP8266WiFi.h>
2 #include <Arduino.h>
3
4 // Struktur data penerima (wajib identik dengan sisi pengirim)
5 typedef struct _struct_pesanan {
6     int paketId;
7     int nilaiParameter;
8 } struct_pesanan;
9
10 struct_pesanan paketIdIsi;
11
12 // callback untuk menerima data dari paket masuk
13 void onDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
14     char macStr[12];
15     sprintf(macStr, "%02x:%02x:%02x:%02x:%02x:%02x",
16             mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
17
18     // Salin buffer memori radio langsung ke struktur data variabel
19     memcpy(&paketIdIsi, incomingData, sizeof(paketIdIsi));
20
21     Serial.print("Paket masuk dari: ");
22     Serial.print(macStr);
23     Serial.print(" Ukuran Data: ");
24     Serial.print(len);
25     Serial.print(" Byte");
26     Serial.print(" Durasi: ");
27     Serial.print(paketIdIsi.paketId);
28     Serial.print(" Nilai Parameter: ");
29     Serial.print(paketIdIsi.nilaiParameter);
30     Serial.println("-----");
31 }
```

Serial Monitor Output:

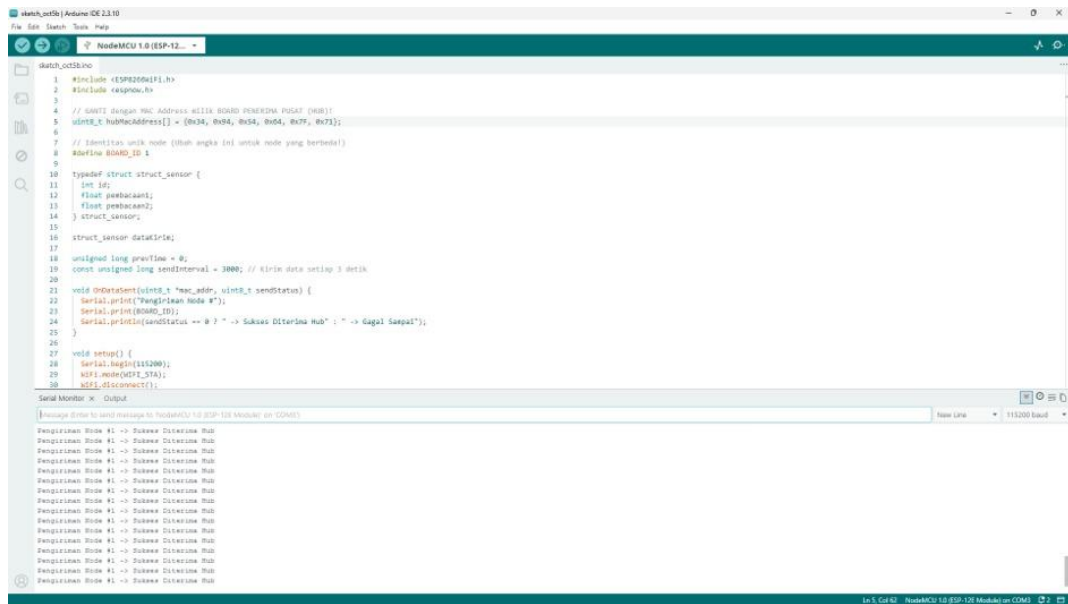
```
=====
Paket masuk dari: 54:54:54:04:77:71
Ukuran Data: 8 byte
Durasi: 100
Nilai Parameter: 05
=====
Paket masuk dari: 54:54:54:04:77:71
Ukuran Data: 8 byte
Durasi: 100
Nilai Parameter: 05
=====
Paket masuk dari: 54:54:54:04:77:71
Ukuran Data: 8 byte
Durasi: 100
Nilai Parameter: 13
=====
```

5.5.3 Praktikum 3: Pengumpulan Data Telemetri Many-to-One (ESP-NOW)

Skenario: Dua node sensor di lapangan memancarkan data pengukuran masing-masing ke sebuah node konsentrator pusat (*Aggregator Hub / Base Station*).

Langkah Kerja Pemrograman Sisi Node Pengirim:

1. Masukkan alamat MAC milik Board Penerima Pusat (Hub) pada variabel **hubMacAddress**.
2. Pada Board Pengirim 1, atur: **#define BOARD_ID 1**.
3. Pada Board Pengirim 2, atur: **#define BOARD_ID 2**.
4. Unggah kode program berikut:



```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // SAKTI dengan Mac Address milik BOARD PENERIMA PUSAT (HUB)
5 uint8_t hubMacAddress[] = {0x34, 0x94, 0x64, 0x64, 0x79, 0x71};
6
7 // Identitas unik node (Ubah angka ini untuk node yang berbeda)
8 #define BAHARU_ID 1
9
10 typedef struct struct_sensor {
11     int id;
12     float pembacaan1;
13     float pembacaan2;
14 } struct_sensor;
15
16 struct_sensor data1;
17
18 unsigned long prevTime = 0;
19 const unsigned long sendInterval = 3000; // Kirim data setiap 3 detik
20
21 void onDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
22     Serial.println("Pengiriman Node #7");
23     Serial.println(BAHARU_ID);
24     Serial.println(sendStatus == 0 ? "Sukses Diterima Hub" : "Gagal Saapet");
25 }
26
27 void setup() {
28     Serial.begin(115200);
29     WiFi.mode(WIFI_STA);
30     WiFi.disconnect();
31 }
```

Serial Monitor X Output

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on COM7)

Parameter 2 : 46.60

DATA TERBARU DARI NODE #1

Parameter 1 : 20.20

Parameter 2 : 57.50

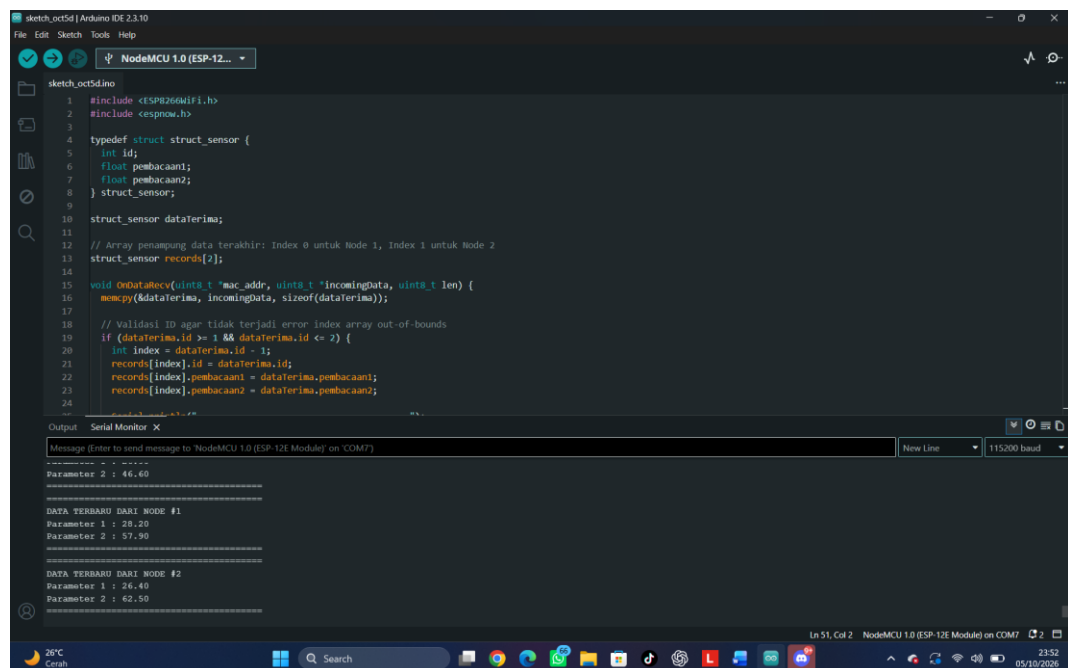
DATA TERBARU DARI NODE #2

Parameter 1 : 26.40

Parameter 2 : 62.50

Langkah Kerja Pemrograman Sisi Hub Konsentrator:

Unggah kode sketsa berikut ke Board Penerima Pusat:



```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 typedef struct struct_sensor {
5     int id;
6     float pembacaan1;
7     float pembacaan2;
8 } struct_sensor;
9
10 struct_sensor data1;
11
12 // Array penampung data terakhir: Index 0 untuk Node 1, Index 1 untuk Node 2
13 struct_sensor records[2];
14
15 void onDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
16     memcpy(data1, incomingData, sizeof(data1));
17
18     // Validasi ID agar tidak terjadi error index array out-of-bounds
19     if (data1.id >= 1 && data1.id <= 2) {
20         int index = data1.id - 1;
21         records[index].id = data1.id;
22         records[index].pembacaan1 = data1.pembacaan1;
23         records[index].pembacaan2 = data1.pembacaan2;
24     }
25 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on COM7)

Parameter 2 : 46.60

DATA TERBARU DARI NODE #1

Parameter 1 : 20.20

Parameter 2 : 57.50

DATA TERBARU DARI NODE #2

Parameter 1 : 26.40

Parameter 2 : 62.50

5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)

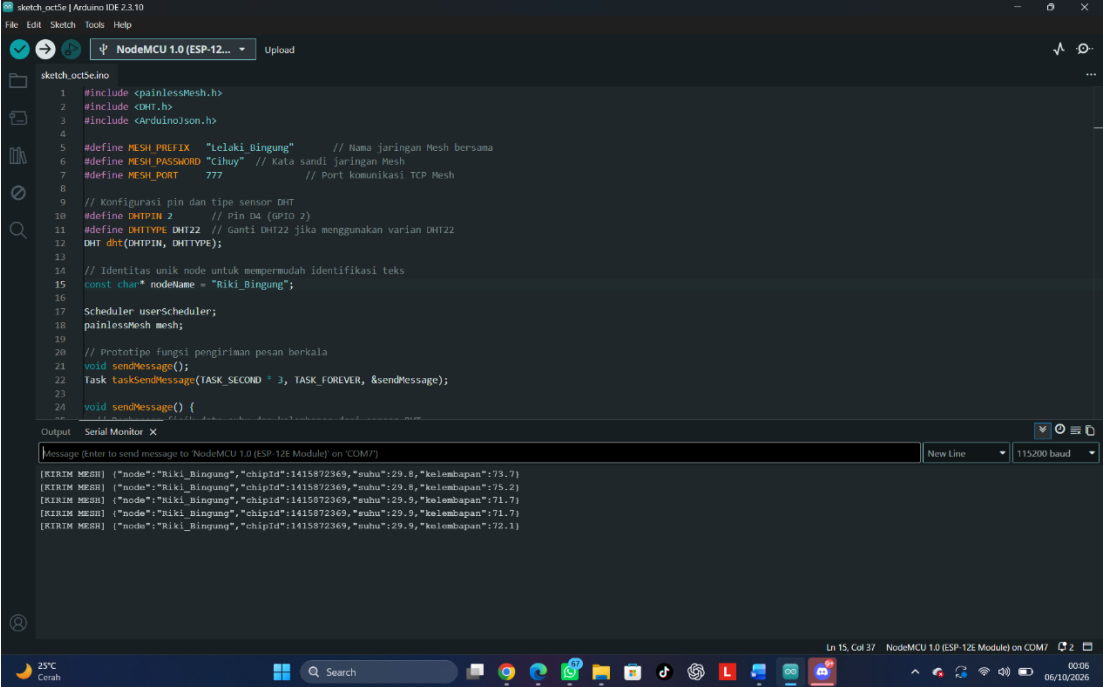
Skenario: Seluruh board dalam kelompok dihubungkan ke sensor suhu & kelembapan riil (DHT11/DHT22) dan diintegrasikan ke dalam sebuah jaringan rantai otonom (*Mesh*). Setiap board secara berkala membaca sensor fisiknya, mengemas data telemetri ke dalam format JSON, dan menyiarkannya ke seluruh anggota jaringan tanpa perlu mengetahui alamat MAC masing-masing.

Instruksi Perangkaian Perangkat Keras:

- Pasang sensor DHT11 / DHT22 pada masing-masing board NodeMCU:
 - Pin VCC sensor ke pin 3V3 (3.3V) NodeMCU.
 - Pin GND sensor ke pin GND NodeMCU.
 - Pin DATA / OUT sensor ke pin D4 (GPIO 2) NodeMCU.

Langkah Kerja Pemrograman:

1. Unggah sketsa berikut ke seluruh board dalam kelompok Anda.
2. Berikan nama pengenalan unik pada variabel nodeName di masing-masing board (misal: "Node-1", "Node-2", "Node-3").
3. Jika menggunakan varian DHT22, ubah #define DHTTYPE DHT11 menjadi DHT22.



```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "Telaki Bingung" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "Cibuy" // Kata sandi jaringan Mesh
7 #define MESH_PORT 777 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT11 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Riki Bingung";
16
17 Scheduler userScheduler;
18 painlessMesh mesh;
19
20 // Prototipe fungsi pengiriman pesan berkala
21 void sendMessage();
22 Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
23
24 void sendMessage() {
25     // ... (code for sending JSON message) ...
26 }
```

Output Serial Monitor:

```
[KTRIM MESH] ("node":"Riki_Bingung","chipid":1415872369," suhu":29.8,"kelembapan":73.7)
[KTRIM MESH] ("node":"Riki_Bingung","chipid":1415872369," suhu":29.8,"kelembapan":75.2)
[KTRIM MESH] ("node":"Riki_Bingung","chipid":1415872369," suhu":29.9,"kelembapan":71.7)
[KTRIM MESH] ("node":"Riki_Bingung","chipid":1415872369," suhu":29.9,"kelembapan":71.7)
[KTRIM MESH] ("node":"Riki_Bingung","chipid":1415872369," suhu":29.9,"kelembapan":72.1)
```

5.6 Latihan

Kerjakan latihan berbasis eksperimen, observasi serial monitor, dan modifikasi kode berikut untuk melengkapi laporan praktikum Anda:

- **Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2):** Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (`sendStatus == 0`), lakukan eksperimen pemutusan simpul:
 - i. Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.
 - ii. Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?
 - iii. Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2 MAC* dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!

Pengamatan Log Serial Monitor: Ya, status pengiriman untuk MAC address board penerima yang dicabut dayanya akan segera berganti menjadi "Gagal (Tidak Terjangkau)" di dalam fungsi *callback* OnDataSent

[illegible]

- Waktu Deteksi Node Keluar (*Dropout*):**

- Waktu Pendaftaran Kembali (*Re-joining*):**

- Saat daya pada board tersebut disambungkan kembali, node akan secara otonom mengenali SSID dan *password* mesh yang sama.
- Waktu yang dibutuhkan hingga node tersebut kembali terdaftar ke dalam topologi mesh berkisar antara 2 hingga 10 detik secara otomatis tanpa perlu mengubah konfigurasi ulang apa pun.

```
sketch_oct14 [Arduino IDE 23.10]
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12E...
sketch_oct14ino
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "Lelaki_Bingung" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "Cihuy" // Kata sandi jaringan Mesh
7 #define MESH_PORT 7777 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GP10-2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodename = "niki_bingung";
16
17 Scheduler usersScheduler;
18 painlessMesh mesh;
19
20 // Prototipe fungsi pengiriman pesan berkala
21 void sendMessage();
22 Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
23
24 void sendMessage() {
25     // ...
26 }
```

Output Serial Monitor X

Message (Enter to send message to "NodeMCU 1.0 (ESP-12E Module)" on "COM7")

New Line 115200 baud

```
[KIRIM MESH] ["node":"niki_bingung","chipId":"1415872369","suha":"30.6","kelembapan":71.6]
[KIRIM MESH] ["node":"niki_bingung","chipId":"1415872369","suha":"30.6","kelembapan":71.5]
[KIRIM MESH] ["node":"niki_bingung","chipId":"1415872369","suha":"30.6","kelembapan":71.4]
[KIRIM MESH] ["node":"niki_bingung","chipId":"1415872369","suha":"30.6","kelembapan":71.3]
[KIRIM MESH] ["node":"niki_bingung","chipId":"1415872369","suha":"30.6","kelembapan":71.3]
r1415872369...
STARTUP: A tcp server established on port 7777
RECEIVED: success
Mesh Node [Riki_Bingung] Berjalan. Menunggu pemantauan topologi...
[KIRIM MESH] ["node":"niki_bingung","chipId":"1415872369","suha":"30.6","kelembapan":71.3]
[KIRIM MESH] ["node":"niki_bingung","chipId":"1415872369","suha":"30.6","kelembapan":71.2]
[KIRIM MESH] ["node":"niki_bingung","chipId":"1415872369","suha":"30.6","kelembapan":71.2]
```

- **Latihan 3 (Eksperimen Modifikasi Filter Telemetry & Ambang Batas JSON - Praktikum 4):** Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetry dari semua pengirim tanpa penyaringan.
 - i. Modifikasilah fungsi `receivedCallback` pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis: program HANYA** mencetak kartu telemetry penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai di atas 32.0 °C (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).
 - ii. Jika suhu bernilai normal (≤ 32.0 °C), program cukup mencetak status singkat satu baris: `[NORMAL]` Telemetry dari Node `<chipId>` aman.
 - iii. Tuliskan blok fungsi `receivedCallback(uint32_t from, String &msg)` hasil modifikasi Anda!

```

142 MESH_PASSWORD,
143 UserScheduler,
144 MESH_PORT
145 );
146
147 // Mendaftarkan callback
148 mesh.onReceive(&receivedCallback);
149 mesh.onNewConnection(&newConnectionCallback);
150 mesh.onChangedConnections(&changedConnectionCallback);
151
152 // Menambahkan task pengiriman data
153 userScheduler.addTask(taskSendMessage);
154 taskSendMessage.enable();
155
156 Serial.printf(
157   "Mesh Node [%s] Berjalan. Menunggu pembentukan topologi...\n",
158   nodeName
159 );
160
161
162 void loop() {
163   // Memperbarui jaringan mesh
164   mesh.update();
165 }
  
```

Output Serial Monitor X

```

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on COM7)
[KIRIM MESH] ("node":"Riki_Bingung","chipId":1415872369,"suhu":30.9,"kelembapan":69.9)
[NORMAL] Telemetry dari Node 1415872369 aman.
[KIRIM MESH] ("node":"Riki_Bingung","chipId":1415872369,"suhu":30.9,"kelembapan":70)
[NORMAL] Telemetry dari Node 1415872369 aman.
[KIRIM MESH] ("node":"Riki_Bingung","chipId":1415872369,"suhu":30.9,"kelembapan":70)
[NORMAL] Telemetry dari Node 1415872369 aman.
[KIRIM MESH] ("node":"Riki_Bingung","chipId":1415872369,"suhu":30.9,"kelembapan":70)
[NORMAL] Telemetry dari Node 1415872369 aman.
[KIRIM MESH] ("node":"Riki_Bingung","chipId":1415872369,"suhu":30.9,"kelembapan":69.9)
[NORMAL] Telemetry dari Node 1415872369 aman.
[KIRIM MESH] ("node":"Riki_Bingung","chipId":1415872369,"suhu":30.8,"kelembapan":70)
[NORMAL] Telemetry dari Node 1415872369 aman.
  
```

Ln 151, Col 1 NodeMCU 1.0 (ESP-12E Module) on COM7 2

24°C Cerah 02:40 06/10/2026